

Post Quantum Shell - PQS 1.1

Revision 1.1, June 2, 2026

John G. Underhill - contact@qrcscorp.ca

This document is an engineering level description of the Post Quantum Shell protocol. This document describes the Post Quantum Shell (PQS), an application-layer remote shell, command, and file-transfer protocol carried inside the encrypted QSMS Simplex transport.

Contents	Page
<u>Foreword</u>	2
1. <u>Introduction</u>	2
2. <u>Protocol Description</u>	3
3. <u>Terms and Definitions</u>	5
4. <u>Cryptographic Primitives</u>	10
5. <u>Protocol Components and State Structures</u>	12
6. <u>Protocol Operational Overview</u>	23
7. <u>Mathematical Description</u>	31
8. <u>Security Analysis</u>	39
9. <u>PQC Cryptanalysis</u>	43
10. <u>Application Scenarios</u>	47
11. <u>Conclusion</u>	50

Foreword

This document is the first revision of the specification of PQS, further revisions may become necessary during the pursuit of a standard model, and revision numbers shall be incremented with changes to the specification. The reader is asked to consider only the most recent revision of this draft, as the authoritative implementation of the PQS specification.

The author of this specification is John G. Underhill, and can be reached at john.underhill@protonmail.com

PQS, the algorithm constituting the PQS messaging protocol is patent pending, and is owned by John G. Underhill and Quantum Resistant Cryptographic Solutions Corporation. The code described herein is copyrighted, and owned by John G. Underhill and Quantum Resistant Cryptographic Solutions Corporation.

1. Introduction

The **Post Quantum Shell (PQS)** is a high security, post-quantum communication protocol designed to replace traditional Secure Shell (SSH) for remote server access and communication. With the advent of quantum computing, classical encryption algorithms like RSA, ECDH, and DSA are at risk of being broken, making quantum-resistant cryptographic techniques essential. PQS leverages post-quantum cryptographic primitives, including Kyber (lattice-based key encapsulation), and digital signatures from Dilithium, which are designed to resist both classical and quantum computing threats.

PQS is particularly well suited for industries and applications where long-term confidentiality is critical, such as financial technology (fintech), government communication, healthcare, and critical infrastructure. By offering enhanced security against quantum attacks, PQS ensures that sensitive communications remain secure for years to come.

2. Protocol Description

PQS uses a one-way server-authenticated QSMS Simplex transport model. The client authenticates the server by verifying a signed ephemeral encapsulation key against a pinned server verification key. PQS then performs application-layer user authentication inside the encrypted QSMS channel. The protocol is therefore composed of a QSMS cryptographic transport layer and a PQS application layer for login, command execution, policy enforcement, sandboxing, known-host handling, and file transfer.

This protocol is versatile and can be used in a wide range of applications, such as client registration on networks, secure cloud storage, hub-and-spoke model communications, commodity trading, and electronic currency exchange—essentially, any scenario where an encrypted tunnel using strong, quantum-safe cryptography is required.

The QSMS transport creates a uniquely keyed encrypted channel for a connected peer. The current PQS console server maintains a single active application session instance and resets its active user state when the remote session disconnects. The encapsulation key material used during QSMS establishment is ephemeral for the session, and provisional secrets are erased after key derivation and confirmation.

The server distributes a public signature verification key to its clients. This key is the client-side trust anchor for the QSMS handshake and is also represented as a fingerprint in the PQS known-hosts database. The public verification key may be distributed during a registration event, pre-embedded in client software, or delivered through another authenticated out-of-band process.

2.1 Implementation Profile

The current implementation separates the cryptographic transport from the remote administration application. QSMS performs server authentication, encapsulation, transcript binding, sequence validation, freshness checking, encryption, and authenticated decryption. PQS receives only decrypted application messages from QSMS and applies user authentication, command policy, shell profile selection, sandbox constraints, transfer-root confinement, and known-host verification.

The PQS application payload begins with a one-byte application message type. The remaining bytes contain a fixed login payload, a command string, a file-transfer path, file data, metadata, status text, or a disconnect/error notice depending on the application message type. The application layer does not change QSMS key exchange messages, QSMS packet flags, or QSMS packet header serialization.

The server implementation uses a console administration model. It provides user, shell, and policy administration modes, key and fingerprint display, known-hosts line generation, sandbox status display, detailed help text, and controlled shutdown. The client implementation provides key and fingerprint display, known-hosts inspection and removal, file get and put, directory listing, directory creation, file removal, help, and session termination.

PQS uses policy-first authorization for remote operations. A user must authenticate successfully before command execution or file transfer is accepted. A policy may deny shell execution, allow only configured commands, force a configured command, or permit raw shell execution according to the assigned privilege class and policy configuration. User-supplied command lines are rejected when they contain shell-control metacharacters that would bypass policy evaluation.

The current file-transfer implementation centralizes path handling and metadata processing in pqsxfer. Transfer paths are bounded, relative, non-traversing paths. Server file access is performed through confined open helpers; POSIX builds use descriptor-relative traversal where available, and Windows builds use canonical confinement and reparse-point checks. Recursive traversal is bounded by a fixed recursion limit and skips symbolic links or reparse points.

3.Terms and Definitions

3.1 Cryptographic Primitives

3.1.1 Kyber

The Kyber asymmetric cipher and NIST Post Quantum Competition winner.

3.1.2 McEliece

The McEliece asymmetric cipher and NIST Round 3 Post Quantum Competition candidate.

3.1.3 Dilithium

The Dilithium asymmetric signature scheme and NIST Post Quantum Competition winner.

3.1.5 SPHINCS+

The SPHINCS+ asymmetric signature scheme and NIST Post Quantum Competition winner.

3.1.6 RCS

The wide-block Rijndael hybrid authenticated symmetric stream cipher.

3.1.7 SHA-3

The SHA3 hash function NIST standard, as defined in the NIST standards document FIPS-202; SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions.

3.1.8 SHAKE

The NIST standard Extended Output Function (XOF) defined in the SHA-3 standard publication FIPS-202; SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions.

3.1.9 KMAC

The SHA3 derived Message Authentication Code generator (MAC) function defined in NIST special publication SP800-185: SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash and ParallelHash.

3.2 Network References

3.2.1 Bandwidth

The maximum rate of data transfer across a given path, measured in bits per second (bps).

3.2.2 Byte

Eight bits of data, represented as an unsigned integer ranged 0-255.

3.2.3 Certificate

A digital certificate, a structure that contains a signature verification key, expiration time, and serial number and other identifying information. A certificate is used to verify the authenticity of a message signed with an asymmetric signature scheme.

3.2.4 Domain

A virtual grouping of devices under the same authoritative control that shares resources between members. Domains are not constrained to an IP subnet or physical location but are a virtual group of devices, with server resources typically under the control of a network administrator, and clients accessing those resources from different networks or locations.

3.2.5 Duplex

The ability of a communication system to transmit and receive data; half-duplex allows one direction at a time, while full-duplex allows simultaneous two-way communication.

3.2.6 Gateway: A network point that acts as an entrance to another network, often connecting a local network to the internet.

3.2.7 IP Address

A unique numerical label assigned to each device connected to a network that uses the Internet Protocol for communication.

3.2.8 IPv4 (Internet Protocol version 4): The fourth version of the Internet Protocol, using 32-bit addresses to identify devices on a network.

3.2.9 IPv6 (Internet Protocol version 6): The most recent version of the Internet Protocol, using 128-bit addresses to overcome IPv4 address exhaustion.

3.2.10 LAN (Local Area Network)

A network that connects computers within a limited area such as a residence, school, or office building.

3.2.11 Latency

The time it takes for a data packet to move from source to destination, affecting the speed and performance of a network.

3.2.12 Network Topology

The arrangement of different elements (links, nodes) of a computer network, including physical and logical aspects.

3.2.13 Packet

A unit of data transmitted over a network, containing both control information and user data.

3.2.14 Protocol

A set of rules governing the exchange or transmission of data between devices.

3.2.15 TCP/IP (Transmission Control Protocol/Internet Protocol)

A suite of communication protocols used to interconnect network devices on the internet.

3.2.16 Throughput: The actual rate at which data is successfully transferred over a communication channel.

3.2.17 UDP (User Datagram Protocol)

A communication protocol that offers a limited amount of service when messages are exchanged between computers in a network that uses the Internet Protocol.

3.2.18 VLAN (Virtual Local Area Network)

A logical grouping of network devices that appear to be on the same LAN regardless of their physical location.

3.2.19 VPN (Virtual Private Network)

Creates a secure network connection over a public network such as the internet.

3.3 Normative References

The following documents serve as references for cryptographic components used by QSTP:

3.3.1 FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable Output Functions:

This standard specifies the SHA-3 family of hash functions, including SHAKE extendable-output functions. <https://doi.org/10.6028/NIST.FIPS.202>

3.3.2 FIPS 203: Module-Lattice-Based Key Encapsulation Mechanism (ML-KEM): This standard specifies ML-KEM, a key encapsulation mechanism designed to be secure against quantum computer attacks. <https://doi.org/10.6028/NIST.FIPS.203>

3.3.3 FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA): This standard specifies ML-DSA, a set of algorithms for generating and verifying digital signatures, believed to be secure even against adversaries with quantum computing capabilities. <https://doi.org/10.6028/NIST.FIPS.204>

3.3.4 NIST SP 800-185: SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash, and ParallelHash: This publication specifies four SHA-3-derived functions: cSHAKE, KMAC, TupleHash, and ParallelHash. <https://doi.org/10.6028/NIST.SP.800-185>

3.3.5 NIST SP 800-90A Rev. 1: Recommendation for Random Number Generation Using Deterministic Random Bit Generators: This publication provides recommendations for the generation of random numbers using deterministic random bit generators. <https://doi.org/10.6028/NIST.SP.800-90Ar1>

3.3.6 NIST SP 800-108: Recommendation for Key Derivation Using Pseudorandom Functions: This publication offers recommendations for key derivation using pseudorandom functions. <https://doi.org/10.6028/NIST.SP.800-108>

3.3.7 FIPS 197: The Advanced Encryption Standard (AES): This standard specifies the Advanced Encryption Standard (AES), a symmetric block cipher used widely across the globe. <https://doi.org/10.6028/NIST.FIPS.197>

3.4 PQS Implementation Terms

3.4.1 QSMS

QSMS is the encrypted transport used by PQS. QSMS provides the Simplex key exchange, packet serialization, sequence validation, timestamp freshness checking, RCS channel encryption, and authenticated decryption.

3.4.2 Application Message

An application message is the plaintext PQS payload carried inside an encrypted QSMS packet. The first byte identifies the PQS operation, and the remaining bytes contain the operation-specific data.

3.4.3 Known-Hosts Database

The known-hosts database maps a host identifier to the SHA3-256 fingerprint of the server public verification key record. It provides host-key continuity and change detection for the PQS client.

3.4.4 Strict Host-Key Checking

Strict host-key checking is a client mode in which an unknown server key is rejected instead of being accepted through a trust-on-first-use prompt.

3.4.5 Transfer Root

A transfer root is the configured directory boundary under which a user may perform PQS file operations. The server constructs a per-user transfer root below the configured global transfer directory.

3.4.6 Sandbox Profile

A sandbox profile defines command-execution restrictions including enablement, command timeout, working directory, environment clearing, optional POSIX run-as user and group, and optional POSIX chroot confinement.

3.4.7 Shell Profile

A shell profile names an execution shell, its type, its path, and the privilege mask allowed to use it. Shell profile resolution is separate from command policy authorization.

3.4.8 Command Policy

A command policy defines whether command execution is denied, restricted to an allow list, forced to a configured command, or allowed as raw shell execution subject to denylists and shell-safety checks.

3.4.9 SCB Verifier

An SCB verifier is the stored passphrase verifier derived from the configured SCB parameters, a per-user salt, the PQS verifier domain label, the user name, and the passphrase-derived hash seed.

4: Cryptographic Primitives

PQS relies on a set of cryptographic primitives designed to provide resilience against both classical and quantum-based attacks. The following sections detail the specific cryptographic algorithms and mechanisms that form the foundation of PQS's encryption, key exchange, and authentication processes.

4.1 Asymmetric Cryptographic Primitives

PQS employs post-quantum secure asymmetric algorithms to ensure the integrity and confidentiality of key exchanges, as well as to facilitate digital signatures. The primary asymmetric primitives used are:

- **Kyber**: An IND-CCA secure lattice-based key encapsulation mechanism that provides secure and efficient key exchange resistant to quantum attacks. Kyber is valued for its balance between computational speed and cryptographic strength, making it suitable for scenarios requiring rapid key generation and exchange.
- **McEliece**: A code-based cryptosystem that remains one of the most established and trusted post-quantum algorithms. It leverages the difficulty of decoding general linear codes, offering a high level of security even against advanced quantum decryption techniques.
- **Dilithium**: A lattice-based digital signature scheme based on that of the underlying MLWE and MSIS problems, that offers fast signing and verification while maintaining strong security guarantees against quantum attacks.
- **Sphincs+**: A stateless hash-based signature scheme, which provides long-term security without reliance on specific problem structures, making it robust against future advancements in cryptographic research.

These asymmetric primitives are selected for their proven resilience against quantum cryptanalysis, ensuring that PQS's key exchange and signature operations remain secure in the face of evolving computational threats.

4.2 Symmetric Cryptographic Primitives

PQS's symmetric encryption employs the **Rijndael Cryptographic Stream (RCS)**, a stream cipher adapted from the Rijndael (AES) symmetric cipher to meet post-quantum security needs. Key features of the RCS cipher include:

- **Wide-Block Cipher Design**: RCS extends the original AES design with a focus on increasing the block size (from 128 to 256 bits) and number of transformation rounds (from 14 to 21 for a 256-bit key, and 30 rounds for a 512-bit key), thereby enhancing its resistance to differential and linear cryptanalysis.
- **Enhanced Key Schedule**: The key schedule in RCS is cryptographically strong using Keccak (cSHAKE), ensuring that derived keys are resistant to known attacks, including algebraic-based and differential attacks. RCS replaces Rijndael's cryptographically-weak key schedule, with a strong post-quantum secure key expansion function.

- **Authenticated Encryption with Associated Data (AEAD):** RCS integrates with KMAC (Keccak-based Message Authentication Code) to provide both encryption and message authentication in a single operation. This approach ensures that data integrity is maintained alongside confidentiality.

The RCS stream cipher's design is optimized for high-performance environments, making it suitable for low-latency applications that require secure and efficient data encryption. It leverages AES-NI instructions embedded in modern CPUs. Optionally, PQS can use AES-GCM for standards based AEAD.

4.3 Hash Functions and Key Derivation

Hash functions and key derivation functions (KDFs) are essential to PQS's ability to transform raw cryptographic data into secure keys and hashes. The following primitives are used:

- **SHA-3:** SHA-3 serves as PQS's primary hash function, providing secure, collision-resistant hashing capabilities.
- **SHAKE:** PQS employs the Keccak SHAKE XOF function for deriving symmetric keys from shared secrets. This ensures that each session key is uniquely generated and unpredictable, enhancing the protocol's security against key reuse attacks.
- **KMAC:** The SHA-3 keyed hashing function (MAC), part of the SHA-3 family of post-quantum resistant hashing functions.

These cryptographic primitives ensure that PQS's key management processes remain secure, even in scenarios involving high-risk adversaries and quantum-capable threats.

4.4 Passphrase Verifier Primitive

PQS uses SCB as the passphrase-hardening primitive for server-side user authentication. A user record stores a salt and verifier, not the plaintext passphrase. The verifier construction binds a domain label and user name into the seed material before SCB evaluation. This prevents equivalent verifier inputs for different user names using the same passphrase under the same implementation domain.

The SCB cost parameters are exposed as build-time configuration macros in pqs.h. The default implementation values are `PQS_CRYPTOPHASH_CPU_COST` and `PQS_CRYPTOPHASH_MEMORY_COST`. These values may be tuned by an integrator without modifying the implementation file. Changing the cost parameters or verifier domain changes the verifier derivation profile and requires a user database migration or verifier reset.

5. Protocol Components and State Structures

5.1 Protocol String

The protocol string in PQS is composed of four key components, each representing a specific cryptographic element used in the secure communication process:

1. **Asymmetric Signature Scheme:** Specifies the signature scheme along with its security strength (e.g., s1, s3, s5) from low to high. Example: dilithium-s3 correlates to the NIST *level 3* security designation (192 bits of post-quantum security).
2. **Asymmetric Encapsulation Cipher:** Defines the asymmetric encryption algorithm and its security strength. Example: mceliece-s5.
3. **Hash Function Family:** The designated hash function used within the protocol, which is set as SHA3.
4. **Symmetric Cipher:** The symmetric cipher used for data encryption, set as the authenticated stream cipher RCS.

The protocol string plays a crucial role during the initial negotiation phase to ensure that both the client and server agree on a common set of cryptographic parameters. If the client and server do not support the same protocol settings, a secure connection cannot be established.

Signature Scheme	Asymmetric Cipher	HASH Function	Symmetric Cipher
Dilithium	Kyber	SHA3	RCS
Dilithium	McEliece	SHA3	RCS
Sphincs+	McEliece	SHA3	RCS

Table 5.1: The Protocol string choices in revision PQS 1.3a.

5.2 Client Key Structure

The client key is a publicly exportable structure that contains the signature verification key and associated metadata. It includes parameters such as the key expiration time, protocol string, public signature verification key, and key identity array.

Parameter	Data Type	Bit Length	Function
Expiration	UInt64	64	Validity check
Configuration	UInt8 array	320	Protocol check
Key ID	UInt8 array	128	Identification
Verification Key	UInt8 array	Variable	Authentication

Table 5.2: The client key structure.

- **Expiration:** A 64-bit unsigned integer indicating the number of seconds since the epoch (01/01/1900) until the UTC time when the key remains valid. If the key has expired, the client must request a new public key from the server.

- **Configuration:** Contains the protocol string that defines the cryptographic parameters. If the protocol string on both hosts does not match, the connection is aborted.
- **Key ID:** A unique identifier for the public verification key, facilitating quick reference on the server.
- **Verification Key:** The public asymmetric signature verification key used for authenticating asymmetric encapsulation keys and data during the key exchange.

The client key can be distributed openly or could be encapsulated using X.509 certificates to create a chain of trust, enhancing its security in diverse environments.

5.3 Server Key Structure

The server key is a private (secret) key retained by the server. It contains all elements of the client key plus an additional parameter, the asymmetric signing key.

Data Name	Data Type	Bit Length	Function
Expiration	Uint64	64	Validity check
Configuration	Uint8 array	320	Protocol check
Key ID	Uint8 array	128	Identification
Verification Key	Uint8 array	Variable	Authentication
Signing Key	Uint8 array	Variable	Signing

Table 5.3: The server key structure.

The inclusion of the signing key in the server key structure allows the server to sign messages during the key exchange, ensuring that data exchanges are authenticated and trusted.

5.4 Keep Alive State

The QSMS transport defines packet flags that can represent transport control conditions. The current PQS application layer does not use a separate PQS keep-alive state machine. Session liveness is principally determined by QSMS transport status, command completion, file-transfer completion, and explicit disconnect or error messages.

Parameter	Data Type	Bit Length	Function
Expiration Time	Uint64	64	Validity check
Packet Sequence	Uint64	64	Protocol check
Received Status	Bool	8	Status

Table 5.4: The keep alive state.

If the transport reports a channel failure, authentication failure, receive failure, or application disconnect, the PQS session state is moved toward closing and the active application user is cleared.

5.5 Connection State

The internal connection state structure stores the critical information needed for PQS operations, including cipher states, sequence counters, and the ratchet key.

Data Name	Data Type	Bit Length	Function
Target	Socket struct	664	Validity check
Cipher Send State	Structure	Variable	Symmetric Encryption
Cipher Receive State	Structure	Variable	Symmetric Decryption
Receive Sequence	UInt64	64	Packet Verification
Send Sequence	UInt64	64	Packet Verification
Connection Instance	UInt32	32	Identification
KEX Flag	UInt8	8	KEX State Flag
Ratchet Key	UInt8 array	512	Symmetric Ratchet
PkHash	UInt8 array	256	Authentication
Session Token	UInt8 array	256	Authentication
ExFlag	UInt8	8	Protocol Check

Table 5.5: The connection state structure.

This data structure ensures secure handling of connection parameters, packet sequencing, and cryptographic states during active communication sessions.

5.8 Client KEX State

The Simplex protocol's client and server state structures focus on one-way authentication, storing essential key exchange data:

Data Name	Data Type	Bit Length	Function
Key ID	UInt8 array	128	Key Identification
Session Token	UInt8 array	512	Verification
Remote Verification Key	UInt8 array	Variable	Asymmetric Authentication
Signature Key	UInt8 array	Variable	Asymmetric Authentication
Shared Secret	UInt8 array	256	Symmetric Key
Verification Key	UInt8 array	Variable	Asymmetric Authentication
Expiration	UInt64	64	Verification

Table 5.7: The Simplex client KEX state structure.

5.9 Server KEX State

The Simplex server state structure stores the asymmetric cipher and signature keys used during the key exchange execution.

Data Name	Data Type	Bit Length	Function
Key ID	Uint8 array	128	Key Identification
Session Token	Uint8 array	512	Verification
Private Cipher Key	Uint8 array	Variable	Asymmetric Encryption
Public Cipher Key	Uint8 array	Variable	Asymmetric Encryption
Signature Key	Uint8 array	Variable	Asymmetric Authentication
Shared Secret	Uint8 array	256	Symmetric Key
Verification Key	Uint8 array	Variable	Asymmetric Authentication
Expiration	Uint64	64	Verification

Table 5.8: The Simplex server KEX state structure.

5.10 PQS Packet Header

The PQS packet header is 21 bytes in length, and contains:

Implementation note: the QSMS serialized header order is flag, message length, sequence number, and UTC timestamp. The same serialized header bytes are used as authenticated associated data for encrypted packets. The header is authenticated by QSMS and is not extended by the PQS application layer.

1. The **Packet Flag**, the type of message contained in the packet; this can be any one of the key-exchange stage flags, a message flag, or an error flag.
2. The **Packet Sequence**, this indicates the sequence number of the packet in the exchange.
3. The **Message Size**, this is the size in bytes of the message payload.
4. The **UTC time**, the time the packet was created, used in an anti-replay attack mechanism.

The message is a variable sized array, up to the QSMS message limit. PQS application messages are carried as plaintext application payloads inside the encrypted QSMS message body after QSMS decryption.

Packet Flag 1 byte	Packet Sequence 8 bytes	Message Size 4 bytes	UTC Time 8 bytes
Message Variable Size			

Figure 5.7: The PQS packet structure.

This packet structure is used by the QSMS key exchange and encrypted communications stream. PQS application messages are encoded inside encrypted QSMS payloads and do not alter the QSMS packet header.

5.11 Flag Types

The following is a list of packet flag types used by PQS:

Flag Name	Numerical Value	Flag Purpose
None	0x00	No flag was specified, the default value.
Connect Request	0x01	The key-exchange client connection request flag.
Connect Response	0x02	The key-exchange server connection response flag.
Connection Terminated	0x03	The connection is to be terminated.
Encrypted Message	0x04	The message has been encrypted by the communications stream.
Exchange Request	0x07	The key-exchange client exchange request flag.
Exchange Response	Server decapsulates and derives transport keys	Server sends an RCS-authenticated confirmation message. A mismatch aborts before application data is accepted.
Establish Request	0x09	The key- exchange client establish request flag.
Establish Response	0x0A	The key- exchange server establish response flag.
Keep Alive Request	0x0B	The packet contains a keep alive request.
Keep Alive Response	0x0C	The packet contains a keep alive response.
Remote Connected	0x0D	The remote host has terminated the connection.
Remote Terminated	0x0E	The remote host has terminated the connection.
Session Established	0x0F	The session is in the established state.
Session Established	QSMS confirmation accepted	The PQS application may process login only after QSMS reaches the established transport state.
Unrecognized Protocol	0x11	The protocol string is not recognized
Asymmetric Ratchet Request	0x12	The packet contains an asymmetric ratchet request.
Asymmetric Ratchet Response	0x13	The packet contains an asymmetric ratchet response.
Symmetric Ratchet Request	0x14	The packet contains a symmetric ratchet request.
Error Condition	0xFF	The connection experienced an error.

Table 5.8: Packet header flag types.

5.12 Error Types

The following is a list of error messages used by PQS:

Error Name	Numerical Value	Description
None	0x00	No error condition was detected.
Authentication Failure	0x01	The symmetric cipher had an authentication failure.
Bad Keep Alive	0x02	The keep alive check failed.
Channel Down	0x03	The communications channel has failed.
Connection Failure	0x04	The device could not make a connection to the remote host.
Connect Failure	0x05	The transmission failed at the KEX connection phase.
Decapsulation Failure	0x06	The asymmetric cipher failed to decapsulate the shared secret.
Establish Failure	0x07	The transmission failed at the KEX establish phase.
Exstart Failure	0x08	The transmission failed at the KEX exstart phase.
Exchange Failure	0x09	The transmission failed at the KEX exchange phase.
Hash Invalid	0x0A	The public-key hash is invalid.
Invalid Input	0x0B	The expected input was invalid.
Invalid Request	0x0C	The packet flag was unexpected.
Keep Alive Expired	0x0D	The keep alive has expired with no response.
Key Expired	0x0E	The PQS public key has expired.
Key Unrecognized	0x0F	The key identity is unrecognized.
Packet Un-Sequenced	0x10	The packet was received out of sequence.
Random Failure	0x11	The random generator has failed.
Receive Failure	0x12	The receiver failed at the network layer.
Transmit Failure	0x13	The transmitter failed at the network layer.
Verify Failure	0x14	The expected data could not be verified.
Unknown Protocol	0x15	The protocol string was not recognized.
Listener Failure	0x16	The listener function failed to initialize.
Accept Failure	0x17	The socket accept function returned an error.
Hosts Exceeded	0x18	The server has run out of socket connections.
Allocation Failure	0x19	The server has run out of memory.

Decryption Failure	0x1A	The decryption authentication has failed.
Ratchet Failure	0x1C	The ratchet operation has failed.

Table 5.9: Error type messages.

5.13 PQS Application Message Types

Value	Name	Purpose
0x10	login_request	Client login request containing a fixed-size user name and passphrase payload.
0x11	login_success	Server login success response.
0x12	login_failure	Server login failure response.
0x20	command_request	Client command execution request.
0x21	response_more	Command-output continuation response.
0x22	response_final	Final command-output response.
0x30	error	PQS application error message.
0x31	disconnect	PQS application disconnect notice.
0x40	file_get_request	Client file-download request.
0x41	file_put_start	Client upload start request.
0x42	file_put_data	Client upload data chunk.
0x43	file_put_final	Client upload finalization metadata.
0x44	file_list_request	Client directory-listing request.
0x45	file_mkdir_request	Client directory creation request.
0x46	file_remove_request	Client file-removal request.
0x47	file_data	Server file data chunk.
0x48	file_final	Server file finalization metadata.
0x49	file_status	File-transfer status message.
0x4A	file_get_recursive_request	Client recursive download request.
0x4B	file_directory_begin	Recursive directory begin marker.
0x4C	file_directory_end	Recursive directory end marker.
0x4D	file_begin	Recursive file begin marker.

Table 5.10: PQS application message types carried inside encrypted QSMS payloads.

5.14 PQS Application Session States

Value	State	Purpose
0x00	none	No PQS application session is active.
0x01	connected	QSMS is connected but no PQS login state has been assigned.
0x02	login_required	The encrypted QSMS channel is active and PQS user authentication is required.
0x03	authenticated	The PQS user has authenticated and command or file operations may be authorized.
0x04	command_active	A command request is currently executing.
0x05	closing	The PQS application session is closing.

Table 5.11: PQS application-layer session states.

5.15 User Database and Login Verifier

The user database stores user records containing a user name, enabled state, privilege level, assigned shell profile, failed-attempt counter, passphrase salt, and passphrase verifier. The implementation validates user names and passphrase lengths before committing a record. The passphrase itself is not stored.

The verifier seed is derived from the PQS verifier domain label, the user name, and the passphrase material. SCB is then initialized with the seed, the per-user salt, and the configured SCB cost parameters. The verifier is compared using the QSC constant-time verification routine. Passphrase buffers and intermediate seed material are securely erased after use.

The login path uses timing-neutral verification for valid, missing, disabled, or locked accounts by evaluating a dummy verifier path when the account record is not usable. This reduces account enumeration through gross timing differences. The persistent failed-attempt counter is enforced; a record that reaches the configured login failure threshold is disabled until an administrator changes the account state.

5.16 Shell Profiles, Policy, and Sandbox

A shell profile defines the executable shell path, shell type, enabled state, and privilege mask. A command policy defines the authorization mode applied to a privilege class. The implemented policy modes are no-shell, restricted, forced, and raw-shell. Restricted mode authorizes only

configured commands; forced mode substitutes a configured command; raw-shell mode allows shell execution subject to explicit denial and safety checks.

The command policy engine extracts the leading command verb for allow-list and deny-list decisions and also rejects user-supplied command lines that contain shell-control metacharacters. This check prevents a policy-approved command verb from being used to carry an additional shell expression in the same command line.

The sandbox profile is mandatory for command execution in the hardened implementation. It provides timeout, working-directory, environment-clearing, POSIX run-as user, POSIX run-as group, and POSIX chroot configuration fields. POSIX execution closes non-standard file descriptors before exec and applies no-new-privileges where available. Windows execution uses controlled handle inheritance, restricted-token creation where available, and job-object limits for child process containment.

5.17 File Transfer State and Confinement

The file-transfer subsystem is implemented in pqsxfer. It provides relative-path validation, payload length extraction, bounded relative-path extraction from application messages, safe local path construction, metadata formatting and parsing, recursive transfer metadata, SHA3-256 file hashing, per-user transfer root creation, symlink or reparse-point checks, and confined read/write open helpers.

Server-side file operations are confined to the authenticated user transfer root. The implementation rejects empty paths, absolute paths, drive-qualified paths, traversal components, and oversized paths. POSIX confined file opening uses descriptor-relative traversal with no-follow semantics where available. Windows confined file opening uses canonical root-prefix validation and rejects reparse-point targets before opening.

Recursive transfer traversal is centralized in pqsxfer_walk_directory. The traversal reports directory-begin, file, and directory-end events through a callback. It applies the shared PQS_XFER_RECURSION_MAX recursion limit and skips symbolic links or reparse points. Client recursive upload and server recursive download use this shared walker rather than maintaining separate OS-specific directory traversal logic in console files.

5.18 Client and Server Configuration Fields

Scope	Field	Purpose
Server	storage-root	Root directory for PQS server databases and transfer roots.
Server	sandbox-enabled	Enables command sandbox enforcement.
Server	sandbox-clear-environment	Clears the inherited environment before command execution.

Server	sandbox-working-directory	Working directory used for sandboxed command execution.
Server	sandbox-run-as-user	POSIX user name used for privilege drop.
Server	sandbox-run-as-group	POSIX group name used for privilege drop.
Server	sandbox-chroot-enabled	Enables POSIX chroot confinement when configured and permitted.
Client	known-hosts	Path to the client known-hosts database.
Client	strict-host-checking	Rejects unknown server keys instead of prompting for trust-on-first-use.

Table 5.12: Current implementation configuration fields relevant to PQS security behavior.

5.19 Function Definitions in pqs.h

Function	Description
pqs_packet_to_stream	Serializes a network packet into a byte stream for transmission.
pqs_stream_to_packet	Deserializes a byte array back into a packet structure.
pqs_public_key_decode	Decodes a public key string into a client verification key structure.
pqs_public_key_encode	Encodes a client key structure into a public key string.
pqs_public_key_hash	Hashes a public key structure and returns the result in a byte array.
pqs_signature_key_deserialize	Decodes a secret signature key from a byte array into a server key structure.
pqs_signature_key_serialize	Encodes a server key structure into a serialized byte array.

5.19.1 Function: pqs_packet_to_stream

Purpose: Serializes a network packet into a byte stream for transmission.

Description: Converts a structured packet, containing fields such as message, sequence number, and timestamp, into a byte stream ready for network transmission. The function ensures proper byte ordering, padding, and integrity checks before sending the data over the network.

5.19.2 Function: pqs_stream_to_packet

Purpose: Deserializes a byte array back into a packet structure.

Description: Converts a byte array received from the network back into a packet structure. It verifies the packet format, validates sequence numbers, timestamps, and other fields to ensure the message is legitimate and intact.

5.19.3 Function: pqs_public_key_decode

Purpose: Decodes a public key string into a client verification key structure.

Description: Converts a serialized public key string (typically in base64 or hexadecimal form) into a structured key format that can be used for cryptographic operations.

5.19.4 Function: pqs_public_key_encode

Purpose: Encodes a client key structure into a public key string.

Description: Converts a structured public key into a string format, often used for transmission or storage in human-readable forms like base64.

5.19.5 Function: pqs_public_key_hash

Purpose: Hashes a public key structure and returns the result in a byte array.

Description: Applies a secure hash function to the public key and returns a fixed-size hash value that can be used for verification or integrity checks.

5.19.6 Function: pqs_signature_key_serialize

Purpose: Encodes a server key structure into a serialized byte array.

Description: Converts a signature key (either private or public) into a byte array for storage or transmission.

5.19.7 Function: pqs_signature_key_deserialize

5.20 Current PQS Library Modules

Module	Implementation responsibility
pqs.h	Application message types, session states, privilege identifiers, client command identifiers, and PQS error codes.
pqsuser	User database, privilege conversion, passphrase verifier generation, timing-neutral verification, and user record persistence.
pqsshell	Shell profile database, shell enablement, default shell selection, and privilege-mask checks.

pqspolicy	Command policy database, allow-list and deny-list operations, forced command handling, privilege assignment, and shell-safety checks.
pqssandbox	Sandbox profile defaults, timeout clamping, working-directory validation, and optional POSIX run-as/chroot configuration.
pqsprocess	Platform-specific command process creation, output callback handling, descriptor or handle inheritance control, timeout handling, and sandbox application.
pqskey	Host-key fingerprinting, fingerprint formatting, known-host find, set, remove, and verify operations.
pqsxfer	File-transfer path validation, metadata parsing, hashing, recursive traversal, confined file opening, and per-user transfer roots.
pqsconfig	Server and client configuration defaults, save, load, and field parsing.
pqslogger	Structured application logging with sanitized fields.

Table 5.13: Current PQS implementation modules.

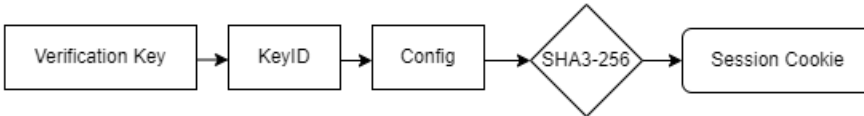
Purpose: Decodes a secret signature key from a byte array into a server key structure.

Description: Converts a serialized byte array containing a signature key back into its structured format to be used in cryptographic operations such as signing or verification.

6 Protocol Operational Overview

6.1 Connection Request

Create the session cookie by hashing the verification key, key-id, and configuration string.



The client sends the connect request message to the server.

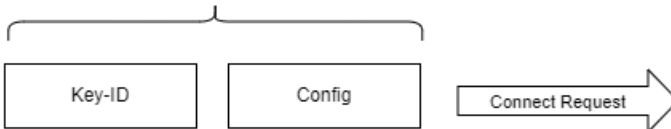


Figure 6.1: QSMS Simplex connection request.

- 1) The client validates the locally configured server public verification-key record before initiating the exchange. The client checks the record expiration time and uses the configured key identity and protocol configuration string as the request payload.
- 2) The client creates a QSMS connect_request packet containing kid and cfg. The serialized QSMS packet header is 21 bytes and is encoded as flag, message length, sequence number, and UTC timestamp.
- 3) The client computes the initial session cookie as $\text{sch} \leftarrow H(\text{cfg} \parallel \text{kid} \parallel \text{pvk})$, where pvk is the pinned server public signature verification key. This value binds the selected configuration, key identity, and server verification key to the exchange transcript.
- 4) The server, not the client, validates the received connect_request header for the expected flag, sequence number, message size, and freshness. The server then checks that kid identifies its configured signing key and that cfg matches its protocol configuration.

6.2 Connection Response

The server receives the connect request from the client.

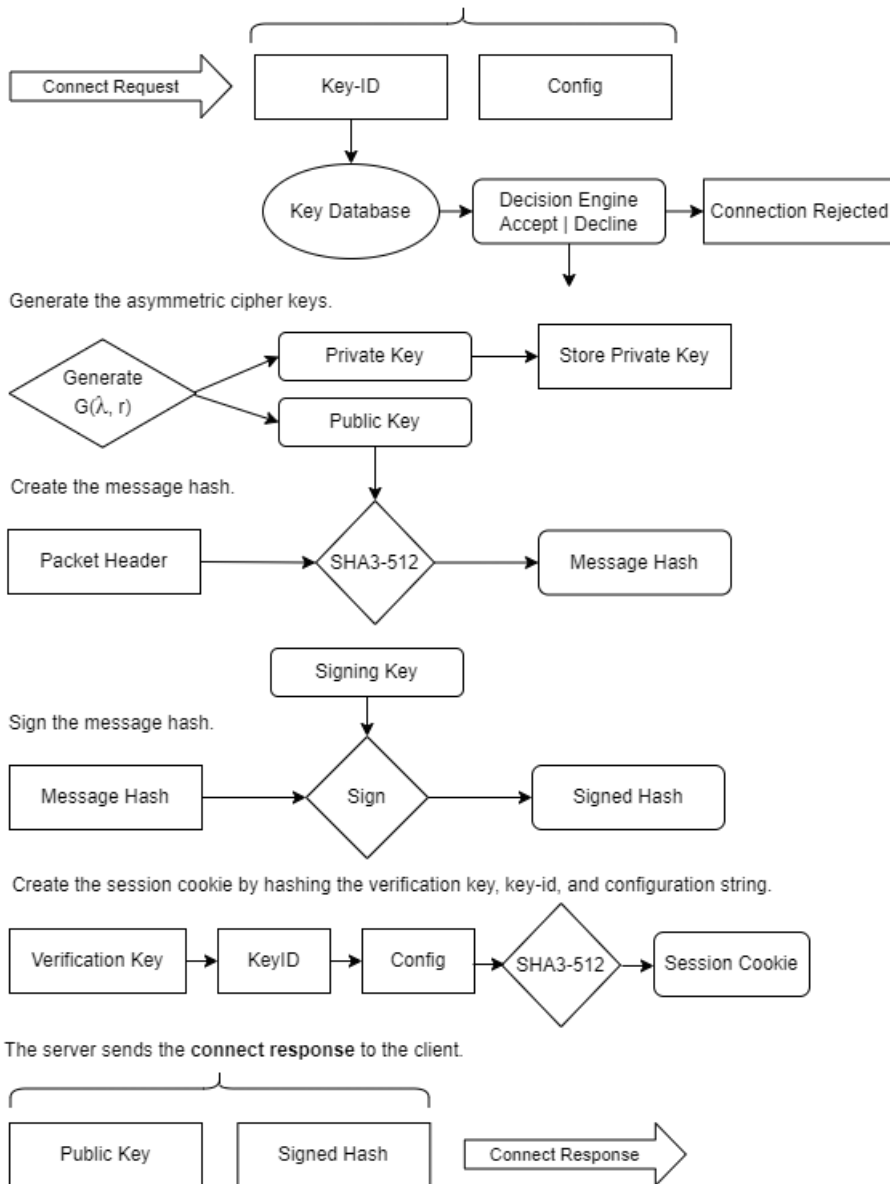


Figure 6.2: QSMS server connection response.

- 1) After a valid `connect_request` is received, the server validates the key identity, configuration string, and key expiration state. Any mismatch causes an error response or connection teardown before key material is generated.
- 2) The server computes the same session cookie, $sch \leftarrow H(cfg \parallel kid \parallel pvk)$, using its configured verification key associated with the selected signing key.
- 3) The server generates a fresh ephemeral KEM encapsulation key pair for the session. The private encapsulation key is retained only in the temporary server key-exchange state.

- 4) The server serializes the connect_response header and computes $\text{phash} \leftarrow H(\text{sh} \parallel \text{pk})$, where sh is the serialized connect_response header and pk is the ephemeral KEM public key.
- 5) The server signs phash with the server signing key and places the signature and ephemeral KEM public key in the connect_response payload.
- 6) The client later verifies the signature using the pinned server verification key and recomputes phash over the exact serialized response header and public key bytes. A failed signature or hash comparison aborts the exchange.
- 7) No PQS application payload is accepted during this phase. The channel remains unavailable until QSMS key confirmation succeeds.

6.3 Exchange Request

The client receives the **connect response** from the server.

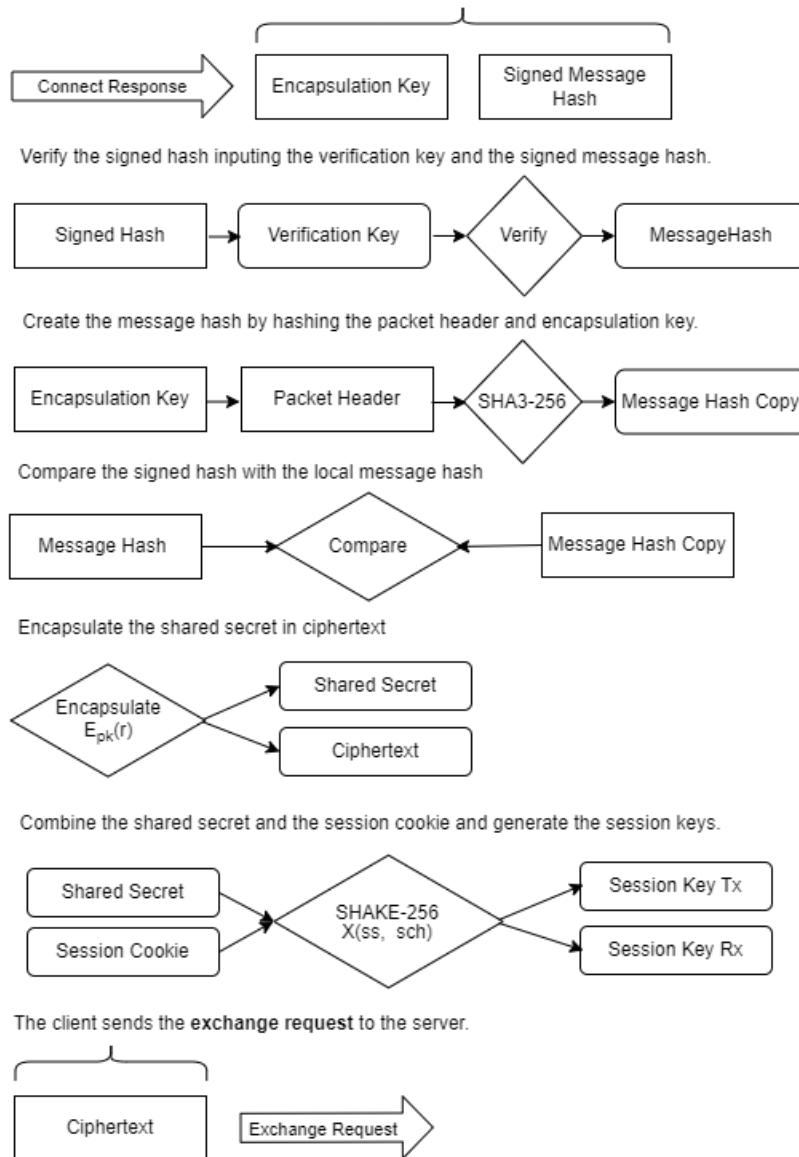


Figure 6.3: QSMS client exchange request.

- 1) The client validates the `connect_response` header for the expected flag, sequence number, expected message size, and valid UTC freshness window.
- 2) The client verifies the server signature under `pvk` and recovers or verifies the signed hash value. The client recomputes $\text{phash} \leftarrow H(\text{sh} \parallel \text{pk})$ using the serialized response header and the received ephemeral public key.
- 3) The client accepts the ephemeral encapsulation key only if the signed value matches the recomputed phash. This check binds the server-authenticated public key to the concrete QSMS response packet.

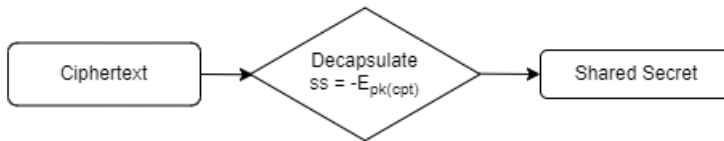
- 4) The client encapsulates to the authenticated ephemeral KEM public key and obtains `cpt` and `sec`. The ciphertext `cpt` becomes the `exchange_request` payload.
- 5) The client derives the QSMS directional key material from the KEM shared secret and session cookie. In implementation terms, cSHAKE is initialized with the shared secret as the keying input and `sch` as customization material.
- 6) The client initializes its receive and transmit RCS cipher states from the derived directional keys and nonces. The client transmit state corresponds to the server receive state, and the client receive state corresponds to the server transmit state.
- 7) The client sends the `exchange_request` containing the KEM ciphertext. Application messages remain unavailable until the authenticated `exchange_response` confirms the derived channel state.

6.4 Exchange Response

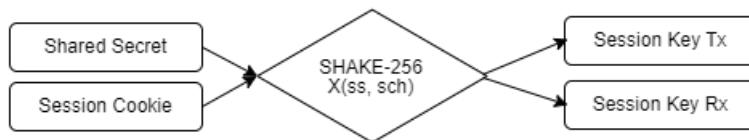
The server receives the **exchange request** from the client.



Decapsulate the clients shared secret.



Combine the two shared secrets and the session cookie and generate the session keys.



The server sends the **exchange response** to the client.



Figure 6.4: QSMS server exchange response.

- 1) The server validates the `exchange_request` header for the expected flag, sequence number, payload size, and freshness before using the KEM ciphertext.
- 2) The server decapsulates `cpt` with the temporary KEM private key and obtains `sec`. Decapsulation failure or malformed input aborts the exchange.
- 3) The server derives the same directional RCS key and nonce material from `sec` and `sch` and initializes the server receive and transmit cipher states.
- 4) The server sends an `exchange_response` packet that is encrypted and authenticated with RCS. The payload carries the expected session confirmation value, and the serialized QSMS `exchange_response` header is supplied as associated data.

6.5 Exchange Response Verification

The client receives the **exchange response** from the server.

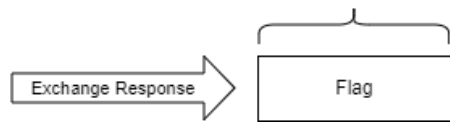


Figure 6.5: QSMS exchange-response verification.

- 1) The client validates the `exchange_response` header and authenticates the packet using the derived RCS receive state with the serialized header as associated data.
- 2) The client accepts the QSMS channel only if RCS authentication succeeds and the decrypted confirmation value matches its locally computed session state. A mismatch causes the client to reject the exchange and tear down the channel.
- 3) After confirmation succeeds, QSMS enters the session-established state. PQS then begins application-layer login inside encrypted QSMS packets; command execution and file transfer remain denied until PQS user authentication succeeds.

6.6 PQS Application Login

After QSMS reports a session-established transport state, the PQS server moves the application state to login-required. The client sends a `login_request` application message containing a bounded user name and passphrase.

The server validates the login payload length, resolves the user record, and verifies the passphrase through the timing-neutral verifier path. Valid, missing, disabled, or locked accounts all use a verification path before a failure response is returned.

A successful login stores the active application user and moves the application session to authenticated. A failed login increments the persistent account failure counter where a matching account exists, and the record is disabled when the configured threshold is reached.

6.7 Command Execution Flow

A `command_request` is accepted only from an authenticated PQS application session. The server resolves the active user, assigned shell profile, applicable command policy, and sandbox profile before execution is considered.

The policy engine applies the configured no-shell, restricted, forced, or raw-shell mode. User-supplied command lines containing shell-control metacharacters are rejected before shell execution so that an allowed verb cannot carry an additional shell expression.

Command execution is delegated to `pqsprocess`. The console server does not implement platform process creation directly; `pqsprocess` applies descriptor or handle inheritance control, environment handling, timeout enforcement, output callbacks, and sandbox restrictions.

6.8 File Transfer Flow

A file operation is accepted only after authentication and policy authorization for the corresponding file verb. The server extracts transfer paths using the authenticated message length and accepts only bounded, NUL-terminated, relative PQS paths.

For download and upload, the server uses the confined read and write helpers in pqsxfer. Recursive transfer uses pqs_xfer_walk_directory, which enforces the recursion bound and refuses symbolic-link or reparse-point traversal.

7. Mathematical Description

Mathematical Symbols

$\leftarrow \leftrightarrow \rightarrow$	-Assignment and direction symbols
$:=, !=, ?=$	-Equality operators; assign, not equals, evaluate
C	-The client host, initiates the exchange
S	-The server host, listens for a connection
$G(\lambda, r)$ source r	-The asymmetric KEM key generation function with parameter set λ and random source r
-Esk	-The asymmetric KEM decapsulation function using secret key sk
Epk	-The asymmetric KEM encapsulation function using public key pk
Ssk	-Sign data with the server secret signature key
Vpk	-Verify a signature using the server public verification key
cfg	-The QSMS protocol configuration string
cprrx	-A receive-channel RCS cipher instance
cprtx	-A transmit-channel RCS cipher instance
cpt	-The KEM ciphertext in the exchange request
ct	-The encrypted QSMS application ciphertext
-Ek	-The symmetric authenticated decryption function and key
Ek	-The symmetric authenticated encryption function and key
H	-The SHA3 hash function
k, n	-A symmetric RCS key and nonce value
KDF	-The cSHAKE-based key expansion function
kid	-The server public-key identity array
Mmk	-The KMAC authentication function and key
pk, sk	-Ephemeral KEM public and secret keys
pvk	-The pinned server public signature verification key
sch	-The QSMS session cookie $H(cfg kid pvk)$
sec	-The shared secret derived by KEM encapsulation and decapsulation

sh, phash, sig -The serialized QSMS packet header, the signed packet hash, and the server signature

QSMS Simplex Key Exchange Sequence

Preamble

The QSMS Simplex key exchange begins with a client that already possesses a server public verification-key record. The client verifies that this local trust anchor is not expired and uses it as the pinned identity for the server-authenticated exchange. PQS application login occurs only after QSMS establishes the encrypted transport channel.

7.1 Connect Request

The client initiates the connection by sending a QSMS connect_request packet containing the server key identity and protocol configuration string.

Key Identity

The key identity (kid) is a 16-byte identifier for the server verification-key record. It selects the server signing key and prevents ambiguity when multiple keys or configurations are provisioned.

Configuration String

The configuration string (cfg) identifies the cryptographic suite expected by both endpoints. If the client-provided cfg does not match the server configuration, the exchange is rejected.

Session Cookie

The client and server both compute the initial session cookie from the configuration string, key identity, and pinned server public verification key:

$$sch \leftarrow H(cfg \parallel kid \parallel pvk)$$

Where:

- *cfg* is the configuration string.
- *kid* is the server key identity.
- *pvk* is the pinned server public verification key.

The session cookie binds the selected QSMS configuration, server key identity, and public verification key to the transcript. It is later used as key-derivation customization material.

The client sends the connect-request payload to the server:

$$C\{kid, cfg\} \rightarrow S$$

7.2 Connect Response

The server processes the connect request and either sends an error packet or a QSMS connect_response packet. Header validation requires the expected flag, sequence number, message size, and UTC freshness window before the request payload is trusted.

Key Verification and Protocol Check

The server verifies that kid identifies its configured signing key and that cfg matches the server protocol configuration.

The server computes the same session cookie:

$$sch \leftarrow H(cfg \parallel kid \parallel pvk)$$

Where:

- *cfg* is the configuration string.
- *kid* is the server key identity.
- *pvk* is the pinned server public verification key.

This value is retained only as key-exchange state and is used to derive the QSMS directional channel keys after KEM decapsulation.

Asymmetric Key Generation and Signing

The server generates a fresh ephemeral KEM key pair and signs a hash that binds the serialized QSMS connect_response header to the ephemeral public key. The signature is generated with the server signing key corresponding to pvk.

Key generation and signing steps:

Generate the ephemeral public and private KEM keys.

$$pk, sk \leftarrow G(\lambda, r)$$

Let sh be the serialized QSMS connect_response header in the implementation order flag || message length || sequence || UTC timestamp. Compute:

$$phash \leftarrow H(sh \parallel pk)$$

Sign the packet-bound public-key hash using the server secret signing key.

$$sig \leftarrow Ssk(phash)$$

The signature covers the response header and the ephemeral KEM public key. It does not create mutual client authentication; client user identity is verified later by the PQS application login protocol.

Server Response

The server sends the signature and ephemeral KEM public key to the client:

$$S\{\text{sig}, \text{pk}\} \rightarrow C$$

7.3 Exchange Request

The client processes the server's connect response and initiates the next steps of the key exchange by verifying the received data, encapsulating a shared secret, and preparing the session keys.

Signature Verification and Hash Check

The client verifies sig using pvk and recomputes phash over the exact serialized connect_response header and the received ephemeral public key. The exchange proceeds only if signature verification succeeds and the recovered or verified value equals the recomputed hash.

The verification relation is:

$$V_{\text{pk}}(\text{sig}) \stackrel{?}{=} H(\text{sh} \parallel \text{pk})$$

After the signed hash check succeeds, the client treats pk as the authenticated ephemeral KEM public key for this session.

The client encapsulates to pk, producing the KEM ciphertext and shared secret:

$$\text{cpt}, \text{sec} \leftarrow E_{\text{pk}}()$$

The client derives directional RCS keys and nonces from sec and sch. In implementation terms, cSHAKE is keyed by sec and customized by sch.

$$k1, k2, n1, n2 \leftarrow \text{KDF}(\text{key} = \text{sec}, \text{custom} = \text{sch})$$

The derived key material is split deterministically into client-transmit/server-receive and client-receive/server-transmit values.

Cipher Initialization

The receive and transmit channel ciphers are then initialized using the derived keys and nonces.

Initialize the client receive channel with the server-to-client key and nonce.

$\text{cprrx}(k2, n2)$

Initialize the client transmit channel with the client-to-server key and nonce.

$\text{cprtx}(k1, n1)$

Client Transmission

The client sends the KEM ciphertext to the server:

$C\{ \text{cpt} \} \rightarrow S$

$C\{ \text{cpt} \} \rightarrow S$

7.3.1 QSMS Key Confirmation Note

QSMS does not use a separate fifth network packet for establish verification in the Simplex path. The server `exchange_response` is the authenticated key-confirmation message. The server encrypts and authenticates the expected confirmation value using the derived RCS transmit state and the serialized QSMS `exchange_response` header as associated data.

7.4 Exchange Response

The server processes the client `exchange_request` by validating the header, decapsulating the KEM ciphertext, deriving the transport keys, and sending the authenticated key-confirmation response.

Shared Secret Decapsulation

The server decapsulates the shared secret from the KEM ciphertext received from the client.

The server uses its ephemeral KEM private key to decapsulate `sec` from `cpt`.

$\text{sec} \leftarrow \text{Esk}(\text{cpt})$

Session Key Derivation

The server derives the directional RCS keys and nonces from the same `sec` and `sch` values used by the client.

The cSHAKE-based KDF is evaluated with the KEM shared secret as keying input and the session cookie as customization material.

$k1, k2, n1, n2 \leftarrow \text{KDF}(\text{key} = \text{sec}, \text{custom} = \text{sch})$

Cipher Initialization

The server initializes the receive and transmit RCS instances with the opposite directional assignment from the client.

Initialize the server receive channel with the client-to-server key and nonce.

$cprx(k1, n1)$

Initialize the server transmit channel with the server-to-client key and nonce.

$cprtx(k2, n2)$

Server Response

The server constructs a QSMS `exchange_response` packet. Its payload is an RCS-authenticated encryption of the expected confirmation value, and the serialized `exchange_response` header is supplied to RCS as associated data.

The server sends the authenticated `exchange_response` to the client. Any key mismatch causes RCS authentication or confirmation comparison to fail at the client.

$S\{ \text{AEAD_RCS}(k2, sh, sch) \} \rightarrow C$

After the response is sent, the server transport state is ready for encrypted QSMS data packets. PQS application messages are still unavailable until the client also accepts the confirmation.

7.5 Exchange Response Verification

The client verifies the authenticated `exchange_response` using the previously derived client receive RCS state. This step is an internal verification of the `exchange_response` packet, not an additional network round trip.

Client Verification

The client validates the `exchange_response` header, supplies the serialized header as associated data, and authenticates the response with RCS. Authentication failure causes immediate exchange failure.

If RCS authentication succeeds, the client compares the decrypted confirmation value with its locally computed session state. The QSMS session is accepted only when the comparison succeeds.

Operational State

After confirmation succeeds, both endpoints treat the QSMS channel as session-established. PQS then sends application messages inside encrypted QSMS packets, beginning with the PQS login request.

7.6 Transmission

During the transmission phase, QSMS carries PQS application payloads inside encrypted_message packets. The first byte of the decrypted payload is the PQS application message type; the remaining bytes are interpreted by the PQS application layer.

Message Serialization and Encryption

The transmitting host serializes the 21-byte QSMS header in the order flag, message length, sequence number, and UTC timestamp. These exact bytes are supplied to RCS as associated data before encryption and authentication.

The message encryption process is as follows:

1. **Encrypt and authenticate the message:** RCS encrypts the plaintext message m and computes an authentication tag over the serialized header and ciphertext.
 $ct, tag \leftarrow Ek(sh, m)$
2. **Associated data binding:** the serialized QSMS header sh is not encrypted, but it is authenticated by the RCS tag. A change to the flag, length, sequence number, or timestamp invalidates the packet.
The authentication function covers sh and ct and produces tag .
 $tag \leftarrow Mmk(sh, ct)$
3. **Append the authentication tag to the ciphertext.** The receiver treats the ciphertext and tag as a single authenticated encrypted payload.

Packet Decryption and Verification

Upon receiving an encrypted packet, the receiver validates the header, expected sequence number, message length, and UTC freshness window before accepting the packet. The serialized header is then supplied to RCS as associated data for authentication.

1. **Verify the tag before releasing plaintext.** The receiver recomputes the authentication value over the serialized header and received ciphertext.
 $tag' \leftarrow Mmk(sh, ct)$
Compare the recomputed tag with the received tag using constant-time verification.
 $tag' \stackrel{?}{=} tag$
If authentication fails, QSMS returns an error and no plaintext is released to PQS. The receive sequence state is not advanced on authentication failure.
2. **Decrypt the ciphertext only after authentication succeeds.**
The ciphertext ct is decrypted back into the plaintext application message m .
 $m \leftarrow -Ek(sh, ct)$

7.7 Application-Layer Login Derivation

Let dom denote the fixed PQS verifier domain string, u denote the user name, p denote the passphrase, salt denote the per-user salt, and SCB denote the configured verifier-hardening function. The user verifier is derived from domain-separated seed material:

$$\begin{aligned}\text{seed} &\leftarrow H(\text{dom} \parallel u \parallel p) \\ \text{verifier} &\leftarrow \text{SCB}(\text{seed}, \text{salt}, \text{cpu_cost}, \text{memory_cost})\end{aligned}$$

The server verifies a login request by recomputing verifier from the submitted passphrase and comparing it with the stored verifier using a constant-time comparison. For an unknown, disabled, or locked account, the implementation evaluates a dummy verifier path before returning failure, reducing timing distinguishability between account states.

7.8 File-Transfer Integrity

Let F be the transferred file byte string and let $hF = \text{SHA3-256}(F)$. File-transfer finalization metadata carries the expected file size and hF as a hexadecimal string. The receiver accepts the completed file only when the received byte count and computed hash match the metadata.

Recursive transfer metadata additionally binds hF to a relative transfer path. The path is interpreted only as a bounded relative PQS path and not as an absolute host file-system path.

This process ensures that QSMS provides confidentiality, header authentication, replay resistance through sequencing, and freshness checking through the UTC timestamp. PQS then applies application-layer login, policy, sandbox, and file-transfer rules to the decrypted payload.

8. Security Analysis

PQS is designed to withstand both classical and quantum attacks, offering future-proof cryptographic security. This section analyzes the protocol's defense mechanisms against common cryptographic threats, focusing on man-in-the-middle attacks, replay attacks, side-channel attacks, and quantum-specific threats. Additionally, we perform a cryptanalysis of the key exchange construction and compare it to other common key exchange protocols.

8.1 Post-Quantum Cryptographic Primitives

PQS relies on post-quantum cryptographic primitives that are resistant to quantum computers, which can break classical cryptography through algorithms like Shor's algorithm and Grover's algorithm. The following primitives are used in PQS:

- **Kyber:** A lattice-based key encapsulation mechanism (KEM) that offers both efficiency and post-quantum security. Kyber ensures that shared session keys are secure against quantum adversaries.
- **Dilithium:** A lattice-based digital signature algorithm used for authentication. Dilithium ensures that digital signatures cannot be forged even with quantum capabilities.

These primitives replace classical algorithms like RSA and ECDH, which are vulnerable to quantum attacks.

8.2 Resistance to Classical Attacks

8.2.1 Man-in-the-Middle (MiTM) Attacks

PQS prevents man-in-the-middle attacks through the use of authenticated key exchange. The key exchange involves signing the server's public key using a post-quantum digital signature (Dilithium) which the client verifies using the server's certificate.

- **Mathematical Defense:** $C_s = \text{Sign}(S_r, P_s), \text{Verify}(P_r, P_s, C_s)$

This guarantees that only the legitimate server's public key is used for key exchange.

8.2.2 Replay Attacks

Replay attacks are mitigated using a **message valid-time check** (UTC_t). Each packet includes a timestamp, and the server only processes messages that fall within a predefined time window. This ensures that even if an attacker intercepts and replays a packet, it will be rejected if it falls outside the valid time window.

- **Mathematical Defense:** Accept if $|\text{UTC}_{\text{received}} - \text{UTC}_{\text{current}}| < \text{Threshold}$

8.2.3 Forward Secrecy

PQS provides **forward secrecy** through the use of **ephemeral key pairs**. During the key exchange, both the client and server generate new public-private key pairs for each session. Once

the session is complete, the ephemeral keys are discarded, ensuring that even if a long-term private key is compromised, past communications remain secure.

- **Mathematical Defense:** The ephemeral session key k is derived for each session:

$$k = \text{KEX}(S_s, P_l) = \text{KEX}(S_l, P_s)$$

- Since S_s and S_l are discarded after use, future key compromises do not affect past session security.

8.2.4 Message Integrity and Authentication

PQS uses **KMAC (Keccak-based Message Authentication Code)** for message integrity and authentication. After encrypting a message, a MAC is generated over the ciphertext to ensure that the message has not been tampered with during transmission. Both the encryption key and the MAC key are derived from the shared session key.

- **Mathematical Defense:**

$$T = \text{MAC}_{k_{\text{auth}}}(M), \text{ Verify } T' = T$$

If the MAC check fails, the message is rejected.

8.3 Resistance to Quantum Attacks

Quantum computers pose a significant threat to classical cryptography by being able to solve problems like factoring large integers or computing discrete logarithms in polynomial time. PQS uses post-quantum cryptographic primitives that are resistant to quantum attacks.

8.3.1 Kyber (Key Encapsulation Mechanisms)

Kyber is used for key encapsulation in PQS, and is the NIST Post Quantum competition winner and standardized for quantum resistant asymmetric ciphers. Kyber is based on the learning with errors (LWE) problem involving finding the shortest path through a lattice. Lattice problems are believed to be resistant to quantum algorithms such as Shor's algorithm.

8.3.2 Dilithium

Dilithium is based on the hardness of finding short vectors in lattices, and is the NIST Post Quantum competition winner and standardized for quantum resistant digital signature schemes.

8.3.3 McEliece

McEliece is a code-based cryptosystem that remains one of the most established and trusted post-quantum algorithms. It leverages the difficulty of decoding general linear codes, offering a high level of security even against advanced quantum decryption techniques.

8.3.4 SPHINCS+

Sphincs+ is a stateless hash-based signature scheme, which provides long-term security without reliance on specific problem structures, making it robust against future advancements in cryptographic research.

8.4 Cryptanalysis of Key Exchange

The PQS key exchange uses a hybrid post-quantum approach, combining classical techniques (like hashing and MACs) with post-quantum encryption and digital signatures. This combination ensures that the key exchange is secure even in the presence of quantum adversaries.

8.4.1 Key Exchange Process

The key exchange in PQS relies on two main operations:

- Key encapsulation using Kyber or McEliece.
- Digital signatures using Dilithium or SPHINCS+.

The server's public key is signed using a post-quantum signature scheme and verified by the client. The client then generates a shared secret using the server's public cipher key and its own private key, ensuring confidentiality and authenticity.

8.4.2 Defense Against Known Attacks

- **Quantum Attacks:** The use of post-quantum algorithms ensures that PQS is secure against both classical and quantum adversaries.
- **Man-in-the-Middle Attacks:** PQS's key exchange mechanism authenticates both the client and server, preventing attackers from intercepting or altering the exchange.
- **Replay Attacks:** The **message valid-time check** prevents an attacker from reusing packets to replay previous communications.
- **Side-Channel Attacks:** The use of constant-time implementations and secure memory management helps mitigate side-channel attacks, where attackers try to exploit information leaks from cryptographic operations.

8.5 Summary of Security Benefits

- **Post-Quantum Security:** PQS's use of McEliece, Kyber, Dilithium, and SPHINCS+ ensures that it is resistant to quantum attacks.
- **Forward Secrecy:** Each session uses ephemeral key pairs, ensuring that past communications remain secure even if long-term keys are compromised.
- **Replay Attack Prevention:** The **message valid-time check** ensures that previously transmitted packets cannot be replayed by an adversary.
- **Man-in-the-Middle Attack Prevention:** PQS authenticates both the server and client, preventing any third party from injecting or altering messages.

- **Resilience Against Side-Channel Attacks:** PQS is designed to resist side-channel attacks through constant-time implementations and secure memory management.

9. Cryptanalysis of the PQS 1.0 Simplex Protocol

9.1 Assumptions, Adversary Model, and Goals

We extend the informal discussion in § 8 by adopting the **Canetti–Krawczyk (CK)** authenticated-key-exchange model specialized to the one-way-trust setting that PQS embraces. The active, adaptive adversary $\mathfrak{A}$

- has full control of the network (eavesdrop, drop, delay, replay, reorder, inject);
- can obtain chosen-ciphertext queries to the IND-CCA KEM (Kyber or McEliece) and chosen-message queries to the EUF-CMA signature (Dilithium or SPHINCS+);
- may compromise long-term server secrets *after* or *before* a run;
- owns a large-scale quantum computer **after protocol termination**.

Target properties:

Label	Informal statement
Server-Auth	Client accepts $\Leftrightarrow$ it verified a Dilithium / SPHINCS+ signature under the server's certified key.
Key-Secrecy	The session key k is indistinguishable from random for $\mathfrak{A}$ unless IND-CCA (KEM) or EUF-CMA (SIG) is broken.
Forward Secrecy (FS)	Compromise of any static signing key after the session leaks no past ciphertext.
Post-Compromise Security (PCS)	Not claimed for the current PQS application data path. Ratchet-related flags are not treated as an implemented PCS mechanism in this specification.
Replay / Downgrade	Sequence number and UTC field inside every MACed header reject stale, reordered, or mixed-version packets.

9.2 Handshake-Level Cryptanalysis

Phase (flag)	Critical check	Result & argument
Connect Req (0x01)	$kid + cfg \rightarrow \text{hash sch} = H(cfg kid pvk)$	Binds protocol string to session; mismatch aborts $\rightarrow$ no silent downgrade .
Connect Resp (0x02)	Server signs $pkh = H(pk hdr) \rightarrow \text{spkh}$	Forgery $\Rightarrow$ EUF-CMA break; hash couples pk to header (inc. timestamp) $\rightarrow$ MitM blocked .
Exchange Req (0x07)	Client verifies spkh , encapsulates sec	If verify fails $\rightarrow$ abort; $\text{cpt} = \text{Enc}_{pk}(\text{sec})$ is IND-CCA $\rightarrow$ k remains secret .

Exchange Resp (0x08)	Server decapsulates and derives transport keys	Server sends an RCS-authenticated confirmation message. A mismatch aborts before application data is accepted.
Session Established	QSMS confirmation accepted	The PQS application may process login only after QSMS reaches the established transport state.

Key-secrecy sketch $k = \text{KDF}(\text{sec} \parallel \text{sch})$, where *sec* is IND-CCA secret and *sch* is public. $\mathcal{A}$ can replace *sch* only by forging the signature in Connect Resp; otherwise the output of **SHAKE** behaves as a PRF, yielding a uniform key.

9.3 Data-Phase Confidentiality & Integrity

The stream cipher **RCS** is keyed with k_{tx} , k_{rx} ; each packet:

1. Serializes a 21-byte header ($\text{flag} \parallel \text{seq} \parallel \text{utc} \parallel \text{len}$).
2. Runs $c = \text{RCS_AEAD}(k, \text{header}, m)$ producing ciphertext $\parallel$ KMAC-tag.
3. Receiver re-computes the tag; mismatch aborts.

Assuming RCS is a PRF-secure wide-block cipher and KMAC is UF-CMA, the channel attains **IND-CPA + INT-CTXT**. Because transmit and receive use *independent* keys, even a tag-forgery oracle on one direction yields no information about the opposite direction.

9.4 Resistance to Standard Attacks

Attack vector	Mitigation in PQS	Residual risk
Man-in-the-Middle	Full-packet signature of server <i>pk</i> ; all subsequent packets MAC'd with fresh key.	None, unless signature or KEM broken.
Replay	seq (64-bit) + utc ($\pm \Delta$ window) inside MAC'd header.	Requires clock sync; widen Δ only under NTP.
Downgrade	cfg hashed into <i>sch</i> ; any mismatch aborts.	Client must pin cfg string it demands.
Key-Compromise Impersonation	Client has no long-term secret; attacker with server's signing key can impersonate server <i>until key rotation</i> .	Mitigated by short-lived certs.
Side channels	The implementation uses constant-time comparison for verifier and authentication checks and securely erases sensitive temporary material where required.	Side-channel resistance of primitive implementations remains dependent on the selected QSC primitive backend and platform build configuration.

Quantum adversary	The selected KEM and signature profile are intended to be post-quantum, and SHA-3 family functions provide the hash, KDF, and MAC foundation.	Security level depends on the configured parameter set and must be stated for the selected profile.
--------------------------	---	---

9.5 Performance Snapshot

The following values are retained as an implementation performance snapshot and shall be remeasured for each release profile and platform:

Step	Kyber-768/Dilithium-3	McEliece-4608/SPHINCS-256s
Keypair + sign (server)	0.32 ms	4.9 ms
Encaps + verify (client)	0.24 ms	5.1 ms
Decaps (server)	0.15 ms	1.7 ms
Cipher setup (SHAKE+RCS)	0.04 ms	0.04 ms
Total 3-RTT latency	≈ 0.9 ms LAN	≈ 12 ms LAN

Bandwidth overhead: 21-byte header + 32-byte (Kyber) or 64-byte (McEliece) tag per packet; handshake < 2 KiB.

9.6 Comparison with Classical SSH and OpenSSH-Hybrid

Dimension	PQS 1.0	OpenSSH-9.7 (RSA/ECDH)	OpenSSH-ML-KEM (draft)
Post-quantum	Native (Kyber/McEliece, Dilithium)	×	Partial (KEM only)
Authentication	Transport server authentication plus PQS application login	Mutual (optional cert)	Mutual
RTT to first byte	3	4	4
FS / PCS	FS / no PCS claim	FS / no PCS claim	Implementation dependent
Replay defense	seq utc in MAC	seq only	seq only
MitM surface	Signed pk + cfg-hash; no fallback algs	Signature on hostkey only; KEX list suffers downgrade unless disabled	Same as OpenSSH classic

Code footprint (LOC, client)	Implementation dependent	Implementation dependent	Implementation dependent
Throughput (AES-NI)	Benchmark dependent	Benchmark dependent	Benchmark dependent

PQS trades SSH-style mutual public-key authentication and multiplexed channel semantics for a lighter server-authenticated transport plus application login. Compared with a general SSH implementation, PQS has a narrower application surface and stronger replay freshness semantics, but it does not claim SSH wire compatibility, implemented PCS, port forwarding, agent forwarding, or full SSH channel equivalence.

9.7 Conclusion

Under standard hardness assumptions for the configured KEM, signature scheme, SHA3/SHAKE/KMAC, and RCS, the QSMS transport used by PQS delivers a server-authenticated encrypted channel with replay resistance, downgrade resistance within the fixed configuration profile, and forward secrecy against later compromise of the long-term server signing key. The current PQS application layer adds login, command policy, sandboxing, known-host continuity, and confined file transfer. The current implementation does not claim SSH wire compatibility, mutual cryptographic client authentication, or implemented post-compromise recovery.

10. Application Scenarios

As quantum computing technology progresses, many existing cryptographic protocols, particularly those based on classical algorithms like **RSA**, **ECDH**, and **DSA**, will no longer be secure. **PQS** addresses this challenge by offering quantum-resistant cryptographic solutions that ensure long-term security, making it a prime candidate to replace **SSH** and other classical protocols in critical industries. Below are several key areas where PQS could be applied.

10.1 Financial Technology (Fintech)

In the financial technology industry, secure communications are essential for a variety of tasks, including secure online transactions, financial data transfers, and remote access to trading systems. **SSH** is currently widely used to secure communications between remote systems in financial institutions. However, with the looming threat of quantum computers, the existing cryptographic methods (such as **RSA** and **ECDH**) will soon become vulnerable.

PQS can provide a quantum-secure alternative to **SSH** in the following ways:

- **Quantum-Secure Transaction Processing:** Payment gateways and financial servers rely on remote secure shell access to manage systems. **PQS** ensures that even if quantum computing becomes a reality, the transaction channels remain secure.
- **Data Protection in Stock Trading:** Remote trading systems can use **PQS** to prevent unauthorized access or tampering with trading data.
- **Long-Term Data Confidentiality:** Fintech companies that store large amounts of sensitive data, such as transaction records and customer information, will benefit from **PQS**'s forward secrecy and resistance to quantum attacks. This ensures that data remains secure well into the future, even after quantum computers become operational.

10.2 Government and Military Communication

Government and military communications require the highest level of security, especially for classified data and remote access to critical infrastructure. **SSH** is commonly used in secure environments for server management, file transfers, and system monitoring. However, its reliance on classical cryptographic primitives makes it vulnerable to future quantum threats.

PQS offers several advantages for secure government and military applications:

- **Secure Remote Access to Critical Infrastructure:** **PQS** can replace **SSH** for remote management of servers and critical systems that control infrastructure such as electricity grids, water supply systems, and communication networks.
- **Classified Communications:** **PQS** ensures the confidentiality and integrity of sensitive information, protecting against both classical and quantum attacks. This is particularly crucial for secure messaging and file transfers involving classified government and military data.

- **Post-Quantum Secure Diplomatic Communication:** Diplomatic channels used by embassies and consulates for secure communication with the home country can be upgraded with PQS to prevent future breaches of sensitive information.

By implementing PQS, governments can future-proof their communication infrastructure, ensuring long-term security even as quantum computing develops.

10.3 Healthcare Systems

The healthcare industry is increasingly reliant on remote access solutions for managing medical devices, electronic health records (EHRs), and telemedicine platforms. SSH is widely used to manage and secure these systems. However, the sensitive nature of healthcare data, combined with the long-term requirement to store patient records securely, makes the adoption of quantum-resistant cryptography essential.

PQS can improve the security of healthcare systems in the following ways:

- **Secure Access to Medical Devices:** Remote management of critical medical devices, such as MRI machines, ventilators, and infusion pumps, can be secured using PQS to prevent unauthorized access or tampering.
- **Telemedicine Platforms:** Doctors can use PQS to securely communicate with patients over encrypted channels, ensuring that sensitive medical information is protected.
- **Secure Health Record Storage:** PQS ensures that patient data, including health records and diagnostic results, remain encrypted and safe from quantum-enabled breaches, preserving patient privacy in the long term.

The **US HIPAA** law requires the protection of sensitive patient information. PQS helps organizations comply with these regulations by providing advanced encryption that remains secure against quantum attacks, thus mitigating future risks.

10.4 Critical Infrastructure

Critical infrastructure systems that control national power grids, transportation networks, water supply systems, and industrial control systems require secure remote management solutions. SSH is often used for remote access to these systems, but as the quantum threat becomes more prominent, it is crucial to transition to post-quantum solutions like PQS.

- **Quantum-Secure Power Grid Management:** Power grids, which require constant remote monitoring and control, can benefit from PQS's secure key exchange and authentication to protect against both classical and quantum threats.
- **Secure Transportation Networks:** Autonomous and remotely managed transportation systems, including train control systems and smart highways, can use PQS to ensure that malicious actors cannot compromise their communications.
- **Industrial Control Systems (ICS):** Remote access to ICS, such as SCADA systems used in manufacturing, can be secured with PQS to protect against unauthorized control or sabotage.

By integrating PQS, critical infrastructure can be protected against the vulnerabilities posed by future quantum threats, ensuring national security and system reliability.

10.5 Cloud Service Providers

Cloud computing environments rely heavily on remote shell protocols like SSH to manage data centers, virtual machines, and customer environments. As quantum computers become more capable, securing the communication between cloud providers and their customers will be essential to maintaining trust in cloud-based services.

- **Secure Cloud Data Centers:** PQS can replace SSH in data centers where administrators manage thousands of virtual machines and sensitive data. This will ensure that critical administrative tasks, such as system updates and security patching, are quantum-safe.
- **Post-Quantum Secure Cloud Storage:** Customers that rely on cloud storage for confidential business or personal data can use PQS to securely access and manage their data.
- **Virtual Private Cloud (VPC) Management:** PQS can be used to manage remote access to VPCs, ensuring that customer environments remain secure even in a quantum computing world.

By adopting PQS, cloud service providers can offer quantum-safe communication channels to their customers, ensuring continued trust in their services.

10.6 PQS as a Replacement for SSH

PQS is designed as a post-quantum alternative for controlled remote administration environments. It is not an SSH wire-compatible implementation. Its current strength is a post-quantum server-authenticated transport combined with a smaller application surface, policy-controlled command execution, and native file-transfer confinement. Some key comparisons include:

- **Key Exchange Security:** SSH relies on ECDH and RSA, which are vulnerable to quantum attacks. PQS uses Kyber and Dilithium, offering quantum resistance while maintaining secure key exchange and authentication.
- **Message Authentication:** PQS uses KMAC for message integrity, offering quantum-resistant authentication in place of SSH's HMAC-based approach, which will be vulnerable in a quantum world.
- **Scalability and Performance:** PQS, with its optimized post-quantum algorithms, can provide similar or better performance compared to SSH when used in environments that require high-throughput, secure communication, such as cloud environments or data centers.

PQS is best characterized as a post-quantum remote-administration protocol that can replace SSH in controlled deployments where its operational model, one-way transport trust, application login, policy engine, sandbox model, and file-transfer subsystem satisfy the deployment requirements.

11. Conclusion

The Post Quantum Shell (PQS) is a post-quantum remote-administration protocol built as an application layer over the QSMS Simplex encrypted transport. As quantum computing capabilities evolve, classical remote-access protocols that depend on RSA, finite-field Diffie-Hellman, or elliptic-curve assumptions require migration paths. PQS addresses this requirement by combining post-quantum server authentication and key establishment with application-layer login, command policy, sandboxing, known-host continuity, and confined file transfer.

11.1 Summary of Key Findings

Throughout the analysis, several key advantages of PQS over traditional protocols like SSH have emerged:

11.1.1 Quantum-Resistant Cryptography

The primary strength of PQS lies in its foundation on post-quantum cryptographic primitives. By using Kyber, Dilithium, RCS and Keccak, PQS ensures that its cryptographic operations such as key exchange, encryption, and digital signatures are secure against both classical and quantum adversaries. These algorithms rely on mathematical problems (such as lattice-based, hash-based, and code-based cryptography) that are believed to be hard for quantum computers to solve.

- **Kyber** provides strong protection for key exchange.
- **McEliece** proven code-based key encapsulation alternative.
- **Dilithium** ensures that signatures are quantum-safe and secure.
- **SPHINCS+** strong hash based signature scheme alternative.
- **Keccak (SHA-3)** and **KMAC** provide secure hashing and message authentication.
- **RCS** provides a powerful post-quantum symmetric cipher with AEAD message authentication.

This quantum-resistance makes PQS a highly secure protocol that will remain effective even as quantum computers become a reality.

11.1.2 Enhanced Key Exchange Security

Compared to SSH, which relies on vulnerable ECDH and RSA for key exchange, PQS uses Kyber or McEliece, which are resistant to quantum attacks. The key exchange process in PQS is also authenticated using Dilithium signatures, ensuring that the public keys cannot be forged or compromised.

The **message valid-time check** (previously referred to as UTC_t) and other security mechanisms within PQS ensure protection against replay attacks and man-in-the-middle attacks, providing enhanced integrity and authenticity for every communication session.

11.1.3 Future-Proof Security

PQS is built with future security challenges in mind. Its use of quantum-resistant cryptography ensures that it will continue to provide secure communication channels long after quantum computers become practical. This future-proofing makes PQS an ideal replacement for SSH, particularly in industries where long-term confidentiality and integrity are critical, such as financial technology (fintech), government, military, healthcare, and cloud service providers.

As countries like the United States and the nations of the European Union begin to mandate the use of stronger encryption standards (e.g., 256-bit keys), PQS offers a viable and efficient alternative to classical protocols, which would otherwise struggle with the increased computational load of larger key sizes.

11.1.4 Performance and Scalability

Despite its use of quantum-resistant algorithms, PQS has been designed to remain efficient and scalable. The post-quantum algorithms used in PQS such as Kyber, and RCS are optimized for performance, ensuring that PQS can handle large volumes of concurrent connections in environments like data centers and cloud platforms without introducing significant latency or overhead.

This makes PQS an ideal solution for industries that require high throughput and scalability, such as cloud computing and fintech, where millions of transactions or remote connections need to be securely managed.

11.2 Potential for Wide Adoption

As quantum computing moves from theory to practice, many industries will be forced to transition away from classical cryptographic protocols like SSH and TLS to ensure the security of their communications and data. PQS is well-positioned to replace these older protocols, offering:

- **Quantum-Secure Remote Access:** Industries that rely on remote access to manage infrastructure, such as cloud service providers, government agencies, and critical infrastructure operators, can deploy PQS as a quantum-resistant alternative to SSH.
- **Secure Transactions in Fintech:** Financial institutions can adopt PQS to ensure that online banking, payment systems, and transaction data remain secure against future quantum threats.
- **Long-Term Confidentiality in Healthcare:** Healthcare organizations can use PQS to protect patient data, ensuring compliance with privacy laws like HIPAA and maintaining the confidentiality of sensitive medical records well into the future.

By adopting PQS in environments that match its trust and administration model, organizations can reduce dependence on classical public-key assumptions while retaining remote command and file-transfer capabilities under a stricter policy and confinement model.

11.3 Current Implementation Summary

The current implementation defines PQS as a QSMS-protected application protocol rather than an SSH-compatible wire protocol. QSMS provides the encrypted transport and the server-authenticated key exchange. PQS provides login, command, policy, sandbox, known-host, and file-transfer semantics inside that transport.

The implemented security posture emphasizes a small protocol surface, explicit application message types, restricted command authorization, mandatory sandbox configuration for execution, known-host continuity with optional strict host-key checking, and transfer-root confinement using descriptor-relative or canonical file access controls.

Future work may add richer interactive session semantics, optional mutual cryptographic client authentication, structured typed-command channels, rekey thresholds or ratchet activation, and broader conformance testing. Such mechanisms shall be specified only when they are implemented and tested in the source tree.